MyCodeSchool

Code review — colections-csharp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 2 — 2026-10-17 — sha `a8e794b`

Scop: fix-urile din runda 1 pe `ListaInlantuita<T>` și pasul 2, `ListaTablou<T>` (commit-ul `a8e794b` „code review + lista tablou").

Metodă: build pe .NET 8, `Program.cs` al tău rulat ca atare (`Main` cheamă doar `Pas2`), apoi, pe o copie, `Pas1` și 28 de teste mici pe ambele liste: listă goală, un element, capetele, `null` , `Clear` , `foreach` ne-generic. Codul tău nu a fost atins.

Poarta de compilare: ✅ **0 erori,** ⚠️ **14 warning-uri** (runda 1: 13). „Gata când" de la Pas 1 cere 0.

Pe scurt:

- ✅ **Runda 1 e închisă pe fond:** `GetEnumerator` cu `yield return` , `RemoveAt(0)` cu `return` și garda `>=` , lista goală și lista cu un element nu mai crapă, `Tail` e corect după constructor, `Clear` și `Insert(0)` . Toate parcurgerile folosesc acum o variabilă locală, `Head` nu se mai mișcă (M1 ✅). `using` -urile nefolosite au dispărut (C1 ✅).
- ✅ `Pas2` **e identic cu fișa**, rând cu rând, inclusiv `Capacitate` 4 → 8 → 16 la al 5-lea și al 9-lea `Add` .
- 🔴 Dar `Pas1` nu mai e chemat din `Main` , așa că n-ai văzut că lista înlănțuită dă acum 3 rânduri greșite (B1, B2). Pe deasupra, o linie rămasă din vechiul cod pierde elemente (B3).
- 🔴 `ListaTablou` repetă două greșeli pe care `ListaInlantuita` le-a reparat deja (B4, B6), plus una nouă (B5).

Ieșirea lui `Pas1`, rând cu rând

Rând din fișă	La tine	De ce
`[] Count=0`	`[] Count=0`	✅
`[a, b, c, d, e] Count=5`	`[a, a, b, c, d, e] Count=5`	B1
`True False`	`True False`	✅
`[b, D, f] Count=3`	`[b, b, D, f] Count=3`	B1
`2 -1`	`2 -1`	✅
`True False`	`True False`	✅
`b;D;f;`	`b;D;f;`	✅
`b|D|f`	`b|D|f|`	B2
`[z] z Count=1`	`[z, z] z Count=1`	B1

Prin `ListaTablou`, aceleași 9 rânduri ies corect: același `DemoLista`, două implementări, două ieșiri diferite. Exact asta trebuia să prindă „Gata când" de la Pas 2.

🔴 Critice

B1 — `ListaInlantuita.cs:275–282`: `ToString` scrie primul element de două ori

Dovada: `[a, b, c]` → `[a, a, b, c]`. În `Pas1`: rândurile 2, 4 și 9 din tabel.

Mecanism: `foreach (T item in this)` folosește `GetEnumerator`-ul tău, care pornește de la `Head`. Deci bucla trece deja prin primul element; linia `text += Head.Data` (cea cu `//???`) îl mai adaugă o dată, în față. Ai simțit că ceva nu e în regulă: problema reală era doar virgula din fața primului element.

💡 `ToString` din `ListaTablou` (`:165–180`) e corect. De ce acolo merge? 💡 Indiciul din runda 1 rămâne valabil: `this` e acum un `IEnumerable<T>`, iar `string.Join` pune singur separatorul doar *între* elemente.

B2 — `ListaInlantuita.cs:98`: `AsVector` întoarce un element în plus

Dovada: `[a, b, c]` → tablou de lungime 4, `[a, b, c, null]`. În `Pas1`, rândul 8: `b|D|f|`.

Mecanism: în runda 1, `AsVector` crăpa cu `IndexOutOfRangeException` fiindcă B2 lăsa `Count` mai mic decât numărul real de noduri. `+ 1` a ascuns simptomul, făcând loc pentru nodul nenumărat. Acum `Count` e corect (B2 din runda 1 e reparat), deci locul în plus rămâne gol: `default(T)`, adică `null` pentru `string` și `0` pentru `int`.

💡 Când `Count` e corect, câte locuri trebuie să aibă tabloul? 💡 Regula de reținut: dacă un indice iese din tablou, întrebi întâi cine a numărat greșit și abia apoi mărești tabloul.

B3 — `ListaInlantuita.cs:57`: `Remove` pe ultimul element mută `Tail` la primul nod

Dovada (copie): `[a, b, c]` → `Remove("c")` → `Add("d")` → `[a, d]` în lanț, dar `Count=3`. `IndexOf("b") = -1`: „b" s-a pierdut. Pe `[a, b, c, d]`, același scenariu pierde „b" și „c".

Mecanism: `Add` leagă noul nod de `Tail`. Dacă `Tail` arată spre primul nod, `Tail.Next = nou` taie tot lanțul de după el. Interesant e de unde vine linia: în runda 1, `Head` era cursorul, deci `Tail = Head` însemna „nodul la care am ajuns", adică cel corect. La M1 ai schimbat cursorul în `current` peste tot, dar această linie a rămas cu `Head`. Același cuvânt, alt înțeles: înainte era o variabilă care se mișca, acum e câmpul fix de la începutul listei.

💡 `RemoveAt` face același lucru corect la `:175–178`. Compară cele două blocuri. 💡 Când schimbi rolul unei variabile, caută **toate** aparițiile ei în metodă, nu doar pe cele din buclă.

B4 — `ListaTablou.cs:160–163`: `GetEnumerator`-ul ne-generic aruncă `NotImplementedException`

Dovada (copie): `foreach (object o in (IEnumerable)lista)` → `NotImplementedException` la `:162`. Este B1 din runda 1, reparat în `ListaInlantuita` (`:269`) și repetat în clasa nouă.

Mecanism: o clasă care implementează `IEnumerable<T>` are două `GetEnumerator`-uri, fiindcă `IEnumerable<T>` moștenește `IEnumerable`-ul vechi, fără tip. `foreach` din `Program.cs` merge, pentru că îl găsește pe cel generic. Dar orice cod care vede lista doar ca `IEnumerable` (un `DataSource` din WinForms, API-urile mai vechi din .NET) îl cheamă pe celălalt și crapă.

💡 Linia de care ai nevoie e deja scrisă, la `ListaInlantuita.cs:269`. De ce acolo merge `=> GetEnumerator()`, iar aici nu? Uită-te la cum e declarat cel generic în fiecare clasă.

B5 — `ListaTablou.cs:11`, `:66`: `Capacity` e un câmp separat de tablou

Dovada (copie): 9 × `Add` → `Clear()` → `Capacity=16`, deși tabloul are iar 4 locuri. După încă 5 `Add`, `Capacity=32`, cu 5 elemente. Iar `lista.Capacity = 0` din `Program` → `Add` aruncă `IndexOutOfRangeException` la `:23`.

Mecanism: capacitatea e *deja* scrisă undeva, în `items.Length`. Un câmp separat e o a doua copie a aceleiași informații, care trebuie ținută la zi de mână în fiecare metodă. `Add` și `Insert` o țin, `Clear` nu. Pe deasupra, câmpul e `public`, deci oricine o poate strica din afară.

💡 Fișa, Pas 2, punctul 2: „`Capacitate`: lungimea tabloului din spate (doar `get`)". Dacă e calculată din tablou, mai poate rămâne în urmă?

B6 — `ListaTablou.cs:36`, `:55`, `:137`: `null` în listă = `NullReferenceException`

Dovada (copie): `["a", null]` → `Contains("b")` → NRE la `:55`. `[null, "a"]` → `IndexOf("a")` → NRE la `:137`. Cele 3 warning-uri `CS8602` din build arată exact aceste trei linii.

Mecanism: `items[i].Equals(item)` cheamă o metodă *pe* element; dacă elementul e `null`, nu are pe ce s-o cheme. `ListaInlantuita` nu are problema, fiindcă folosește `EqualityComparer<T>.Default.Equals(a, b)`, care tratează singur `null` de ambele părți. Este aceeași regulă, respectată într-o clasă și încălcată în cealaltă.

💡 Ce ai scris în `ListaInlantuita.Contains` și nu ai scris în `ListaTablou.Contains`?

🟡 Importante

- **M1** (fost M2) — numele din contract (`Add/Remove/...` în loc de `Adauga/Sterge/...`): rămâne de decis la ședință. Până atunci, `Pas1()` / `Pas2()` există în `Program.cs`, dar `Main` **le cheamă doar pe ultima**. Fișa le cheamă pe toate, una după alta, tocmai ca un pas nou să nu strice unul vechi fără să observi. Așa au trecut B1 și B2.
- **M2** (fost M3) — `Node<T>` e tot `public`, cu câmpuri publice, și sunt 14 warning-uri (9 în `ListaInlantuita`, 3 în `ListaTablou`, 2 în `Node`). Pas 1 nu e „gata" până când nu sunt 0 și nodul nu e `private` în listă (TEORIE §1e: `Node<T>?`).
- **M3** (fost M4) — indexul greșit, încă tăcut, iar acum fiecare clasă tace altfel. `ListaTablou[1] = "x"` pe `[a]` scrie într-un loc pe care nu-l vede nimeni (`Count` rămâne 1), `[1]` întoarce `null`, iar `[-1]` aruncă `IndexOutOfRangeException` (`:148–149`, fără verificare față de `count`). `ListaInlantuita[-1]` întoarce `null`. `ListaInlantuita.Insert(5, x)` pe lista goală inserează oricum, fiindcă ramura `Head == null` (`:114–119`) e înaintea gărzii (`:121`). Pas 6 le aduce pe toate la aceeași `ArgumentOutOfRangeException`.
- **M4** — `ListaTablou`: codul de dublare e scris de două ori (`Add :17–27`, `Insert :88–98`), iar `Insert` / `RemoveAt` caută cu o buclă (`:100–112`, `:119–130`) poziția pe care o primesc deja ca parametru. 💡 O metodă privată pentru „mărește dacă e plin" și o singură buclă de mutare, de la `count` în jos până la `index`.

🟢 Cleanups

- **C1** — constructorul pune încă în `Tail` un nod gol (`ListaInlantuita.cs:14`), iar `Node()` (`Node.cs:8–11`) există doar pentru el. Nu mai strică nimic, fiindcă `Add` întreabă întâi de `Head`, dar „lista goală" e acum descrisă în trei feluri: nod gol, `null` în `Clear`, nod vechi după ultimul `Remove`.
- **C2** — `ListaTablou.Remove` / `RemoveAt` nu golesc locul eliberat (`items[count] = default`): după `Remove("a")` pe `[a, b]`, tabloul ține tot „b" pe poziția 1. Este un detaliu de memorie, pe care `List<T>` din .NET îl face.

- **C3** — `//ICollection`, `//IList`, `//IColectie`, `//ILista`, `//???` și demo-ul `Culori()` comentat în `Main`: notițele merg în `RASPUNSURI.txt`, care încă lipsește.

Q&A

1. În runda 1, `Tail = Head` din `Remove` era corect. N-ai atins linia, și acum e greșită. Ce s-a schimbat în jurul ei?
2. Ce bug a ascuns `new T[Count + 1]` în runda 1 și ce arată acum, după ce acel bug e reparat?
3. `foreach` din `Program.cs` merge pe `ListaTablou`. Atunci cine cheamă `GetEnumerator`-ul ne-generic și când?

Răspunsurile (și cele 3 din runda 1) în `RASPUNSURI.txt`, în repo.

Următorul pas: B1–B6, apoi `Main` cheamă `Pas1()` și `Pas2()` una după alta, ambele identice cu fișa, 0 warning-uri. Abia apoi Pas 3, `ColectieAbstracta`, care va muta oricum `ToString`, `Contains` și `AsVector` într-un singur loc.

Runda 1 — 2026-10-16 — sha `ac84a76`

Pasul 1: `IColectie<T>`, `ILista<T>`, `ListaInlantuita<T>`.

- ✅ **B1** `foreach` / `GetEnumerator`: închis · ⚠️ `ToString` scris, dar dublează primul element (→ runda 2, B1)
- ✅ **B2** `RemoveAt(0)` scădea `Count` de două ori: închis · ⚠️ ascunderea lui prin `Count + 1` în `AsVector` a rămas (→ runda 2, B2)
- ✅ **B3** `Tail` după constructor, `Clear`, `Insert(0)`: închis · 🔴 o linie nouă greșită în `Remove` (→ runda 2, B3)
- ✅ **B4** listă goală și listă cu un element: închis
- ✅ **M1** `Head` folosit drept cursor: închis
- ⏸ **M2** numele din contract: de decis la ședință (→ runda 2, M1)
- 🔴 **M3** nod public + warning-uri: neatins, 13 → 14 (→ runda 2, M2)
- 🔴 **M4** index greșit tăcut: neatins (→ runda 2, M3)
- ✅ **C1** `using`-uri nefolosite · ⚠️ **C2** `Node()` rămas · 🔴 **C3** notițe în cod, `RASPUNSURI.txt` lipsă